

Discrete Variables as Predictors

One often encounters variables which are discrete quantities, but still are on the ratio scale.

$$Y = \beta_0 + \beta_1 X + \epsilon$$


A primary example is that of a **count variable**, i.e., a variable which counts the number of times something occurs. For example, it is standard for credit-granting entities to build models where the response variable is

- the likelihood someone will payoff a loan

and the predictors utilized include

- number of major derogatory reports on credit record
- number of dependents
- number of major credit cards held
- number of active credit accounts

Such predictors **can** be included as predictors just as continuous quantites are; the interpretation remains that same: The expected response increases at a constant rate as the variable increases.

Categorical Predictors

We just discussed using a discrete variable on the ratio scale as a predictor. There are some discrete variables for which this is clearly not a good idea, however. These are cases where the numbers are merely labels for the levels of a **categorical variable**.

Consider a variable which indicates highest degree obtained:

Code	Value
0	Did not finish high school
1	Finished high school
2	Some college
3	Graduated college

Exercise: This variable is on the ordinal scale. What would happen if we mistakenly treated this as a variable on the ratio scale when including it in the regression model?

The effect of going from "Finished HS" to "Some College" is the same as going from "Some college" to "graduated from college", e.g.

Here is the tricky part: A categorical variable with k levels adds $k - 1$ terms into the regression function.

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \epsilon$$

To see why this is the case, imagine that the model under consideration includes a predictor giving the number of credit cards, along with this education level predictor. The $k - 1$ new terms correspond to “shifts” in the regression function relative to the “baseline” category. In particular, let X_1 equal the number of credit cards. Then,

when education level is...	the regression function is...
→ 0 baseline	$\beta_0 + \beta_1 X_1$
1 ←	$\beta_0 + \beta_1 X_1 + \beta_2$
2 ←	$\beta_0 + \beta_1 X_1 + \beta_3$
3 ←	$\beta_0 + \beta_1 X_1 + \beta_4$

We can write this regression function more compactly as

$$\beta_0 + \beta_1 X_1 + \beta_2 \mathbf{1}_{\{\text{edu} = 1\}} + \beta_3 \mathbf{1}_{\{\text{edu} = 2\}} + \beta_4 \mathbf{1}_{\{\text{edu} = 3\}}$$

where $\mathbf{1}_A$ is the **indicator variable** for the event A .

Exercise: Why not include every discrete predictor as being categorical?

Remember a categorical variable with k levels adds $k-1$ parameters. More parameters means more data needed to estimate well. Also, greater risk of overfitting.

- 1 An appropriately-defined categorical variable will be handled correctly by
 2 software such as Python and R.

3 Coefficients:

4	Estimate	Std. Error	t value	Pr(> t)	
5 (Intercept)	35.2647	0.7990	44.137	< 2e-16	***
6 NumCards	-2.1467	0.1915	-11.209	< 2e-16	***
7 (Education)1	5.4004	0.8424	6.411	5.58e-09	*** ←
8 (Education)2	7.8708	0.9308	8.456	3.26e-13	*** ←
9 (Education)3	9.5610	0.8292	11.530	< 2e-16	*** ←

- 10 **Exercise:** Consider the sample output above. What is the expected re-
 11 sponse for a case when education level is "Did not finish high school?"
 12 What about when education level is "Some college?"

13 Did not finish HS: $\hat{\beta}_1 X_1 + \hat{\beta}_0$

14 Some college: $\hat{\beta}_0 + \hat{\beta}_1 X_1 + \hat{\beta}_3$

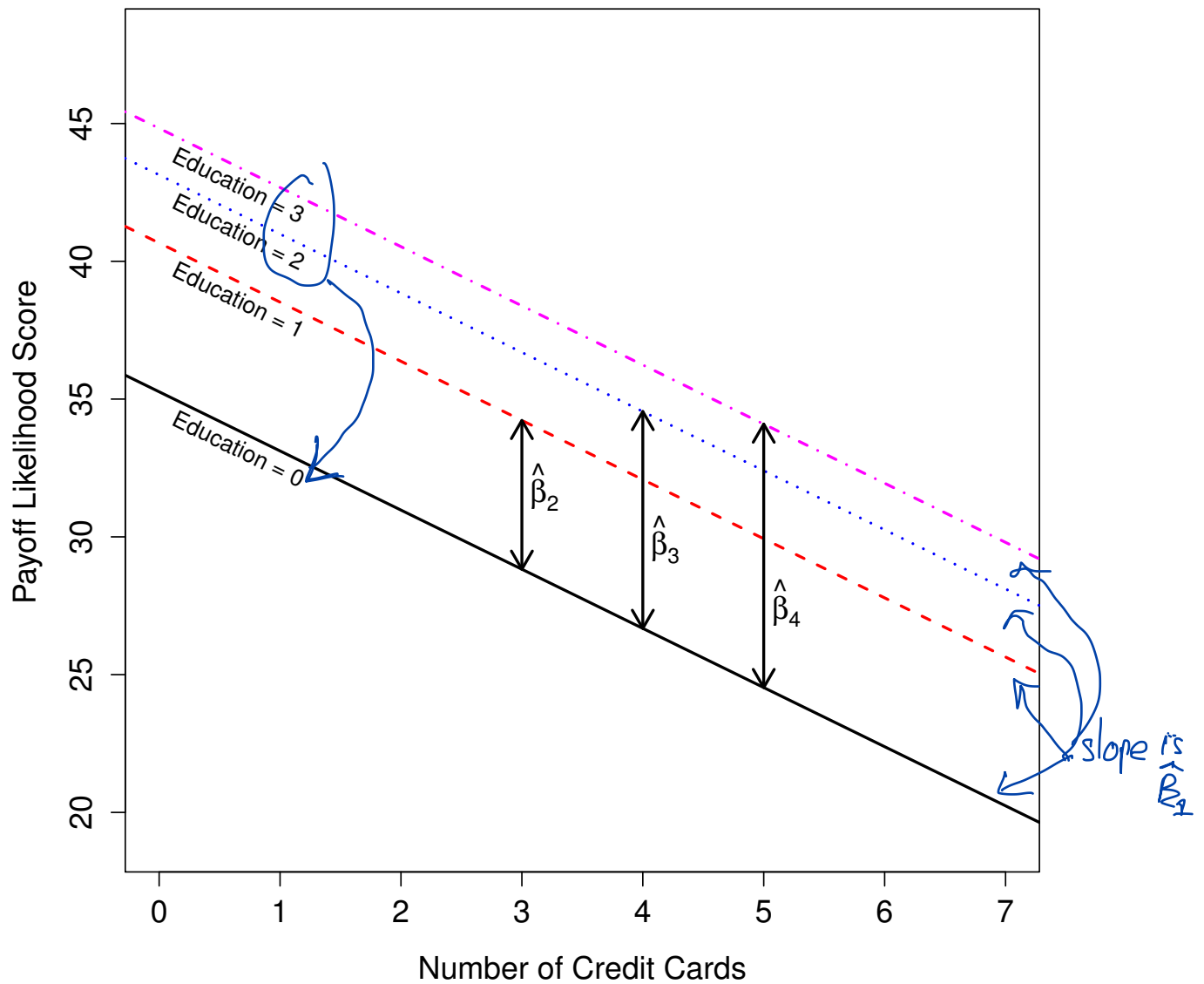


Figure 2: The fit model in this case. The four lines correspond to the four levels of the Education predictor. The slope of each line is $\hat{\beta}_1 = -2.15$.

Categorical Variables in the Design Matrix

Remember that a categorical variable with k levels adds $k - 1$ β parameters to the model. Hence, it adds $k - 1$ **columns** to the design matrix $\mathbf{X}$. In order to create the “shifting” effect, these columns will be filled with zeros and ones.

Consider the following example. The response is the assessment of the likelihood of default. There is a predictor which counts the number of open credit cards. There is another predictor “education level,” defined as above. The first few rows of the data set appear as follows:

Observation	Payoff Likelihood	Number of Cards	Education Level
1	40	4	2
2	41	2	3
3	44	1	2
4	28	5	0
5	36	4	1
6	32	2	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Assume that the analysis proceeds with the predictor “Number of Cards” untransformed.

1 The first six rows of the design matrix would be

$$\mathbf{X} = \begin{matrix} \rightarrow & \begin{matrix} \beta_0 & \beta_1 & \beta_2 & \beta_3 & \beta_4 \end{matrix} \\ \begin{bmatrix} 1 & 4 & 0 & 1 & 0 \\ 1 & 2 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 5 & 0 & 0 & 0 \\ 1 & 4 & 1 & 0 & 0 \\ 1 & 2 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix} \end{matrix}$$

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \beta_3 \\ \beta_4 \end{bmatrix}$$

3 **Exercise:** Explain the form of the design matrix above. Consider the matrix
4 product $\mathbf{X}\beta$.

$$\underline{\underline{\mathbf{X}}} \underline{\underline{\beta}}$$

5 The i^{th} row of $\underline{\underline{\mathbf{X}}} \underline{\underline{\beta}}$ will be the
6 inner product of i^{th} row of $\underline{\underline{\mathbf{X}}}$ and
7 the $\underline{\underline{\beta}}$ vector
8

9 This inner product will capture the appropriate
10 "shift" in the expected response.
11
12
13
14
15

More than One Categorical Variable

Exercise: Suppose that a model contains only two predictors, and they are both categorical variables. The first (X_1) has three levels, and the second (X_2) has four levels. How many total β parameters are in the model, and how is each interpreted?

X_1 has levels a, b, c X_2 has levels $0, 1, 2, 3$

reg. function

β_0

$X_1 = a, X_2 = 0$ (both baseline)

$\beta_0 + \beta_1$

$X_1 = b, X_2 = 0$

$\beta_0 + \beta_2$

$X_1 = c, X_2 = 0$

$\beta_0 + \beta_3$

$X_1 = a, X_2 = 1$

$\beta_0 + \beta_1 + \beta_3$

$X_1 = b, X_2 = 1$

$\beta_0 + \beta_2 + \beta_3$

$X_1 = c, X_2 = 1$

$\beta_0 + \beta_4$

$X_1 = a, X_2 = 2$

$\beta_0 + \beta_1 + \beta_4$

$X_1 = b, X_2 = 2$

$\beta_0 + \beta_2 + \beta_4$

$X_1 = c, X_2 = 2$

$\beta_0 + \beta_5$

$X_1 = a, X_2 = 3$

$\beta_0 + \beta_1 + \beta_5$

$X_1 = b, X_2 = 3$

$\beta_0 + \beta_2 + \beta_5$

$X_1 = c, X_2 = 3$

5 additional β parameters = $2 + 3 = \#$ of levels for $X_1 + \#$ of levels of $X_2 - 1$

Interactions

Interaction terms are added to the model by taking the product of other predictors.

In the absence of any interactions, we say that the model is **additive** in the predictors.

Exercise: Think carefully about what is being assumed when an additive model is used. In particular, what is the effect on the regression function if a covariate is varied?

Assume $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon$

Then, the effect of increasing X_1 by a
is to increase the expected response by $\beta_1 a$,
regardless of the value of X_2

"The model is additive in X_1 and X_2 "

Consider a model with two predictor variables X_1 and X_2 , but with the interaction term X_1X_2 included. So, the regression function is

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 \underline{X_1 X_2}$$

$$\beta_0 + \beta_1 X_1 + (\beta_2 + \beta_3 X_1) X_2$$

Then, if X_1 is held fixed, the effect of varying X_2 is characterized by the slope of $\beta_2 + \beta_3 X_1$. In other words, if X_2 is increased by c , then the regression function increases by $(\beta_2 + \beta_3 X_1)c$.

The key enhancement is that this slope now depends on X_1 .

Of course, one may wonder: Why does it make sense to let the slope depend on X_1 **in this particular manner**?

The best way to answer this is to consider the interaction X_1X_2 to be a **first order approximation** to this dependence.

It is much like how we often use straight lines to approximate relationships which are not truly straight: These are useful approximations in the right circumstances.

Exercise: What will happen if you include in the model an interaction between a ~~continuous~~ ^{on the ratio scale} predictor and a categorical variable?

Assume X_1 is on the ratio scale. X_2 is categorical with levels a, b, c . Include $X_1, X_2, X_1 X_2$.

Output Includes

intercept

X_1

$X_2 b$

$X_2 c$

$X_1 X_2 b$

$X_1 X_2 c$

$\hat{\beta}_0$

$\hat{\beta}_1$

$\hat{\beta}_2$

$\hat{\beta}_3$

$\hat{\beta}_4$

$\hat{\beta}_5$

without interaction, would have these 4

shifts in slope on X_1

What is the effect of increasing X_1 by k ?

When $X_2 = a$, the expected response increases by $\hat{\beta}_1 k$
 " $X_2 = b$, " " " " " $(\hat{\beta}_1 + \hat{\beta}_4)k$
 " $X_2 = c$, " " " " " $(\hat{\beta}_1 + \hat{\beta}_5)k$

Part 9: Power Transformations

In this brief part we will consider families of transforming a sample with the explicit objective of maximizing the quality of the fit of a normal distribution to that sample.

The motivations for such a transformation include (1) to make a skewed distribution more symmetric and “nice”; (2) to allow for the use of the normal model in a case where it otherwise may not be clear how to model the variable; and (3) to transform the response of a regression model so that the normal approximation is a better fit for the model error.

Box-Cox Transformation

This is **only** useful in the case where the ^{variable}~~response~~ is **strictly positive**.

Define Box-Cox **power transformation** function:

$$y^{(\lambda)} = \begin{cases} \frac{y^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \quad \leftarrow y^\lambda \\ \log(y) & \text{if } \lambda = 0 \quad \leftarrow \end{cases}$$

This defines a wide range of possible transformations for the response, including $\sqrt{y}$, y^2 , $\log(y)$, $1/\sqrt{y}$, and so forth.

Exercise: Why is the case $\lambda \neq 0$ defined in the manner it is, instead of just as y^λ ?

Note that

$$\lim_{\lambda \rightarrow 0} \frac{y^\lambda - 1}{\lambda} = \log(y)$$

The family of transformation is continuous
in λ

In Python, this is implemented via the function `boxcox()`, which is part of the package `scipy.stats`.

```
In [1]: from scipy import stats
        import matplotlib.pyplot as plt
        import numpy as np
```

Let's start with a synthetic example.

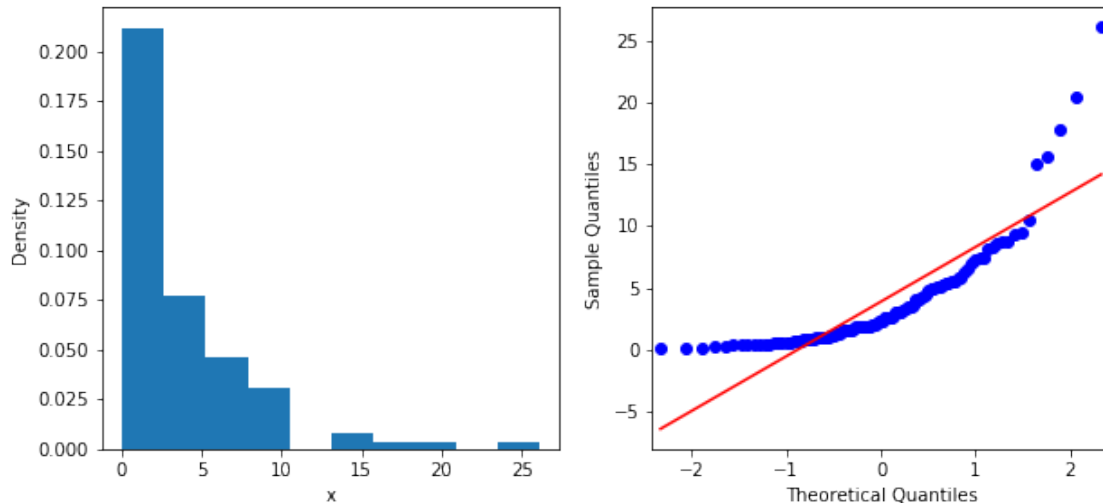
```
In [2]: x = np.random.gamma(1, 4, 100)
```

We note that this is far from a normal shape.

```
In [3]: from statsmodels.graphics.gofplots import qqplot

        fig, axs = plt.subplots(1, 2, figsize=[10, 4.5])
        axs[0].hist(x, density=True)
        axs[0].set_xlabel("x")
        axs[0].set_ylabel("Density")

        qqplot(x, line="s", ax=axs[1])
        plt.show()
```



Now we call the `boxcox` function. By specifying `lambda=None`, we ask the procedure to find the optimal value of λ via maximum likelihood. Specifying `alpha=0.05` means that the procedure will return a 95% confidence interval for λ .

```
In [4]: xt, lam, CI = stats.boxcox(x,
                                     lambda=None, alpha=0.05)

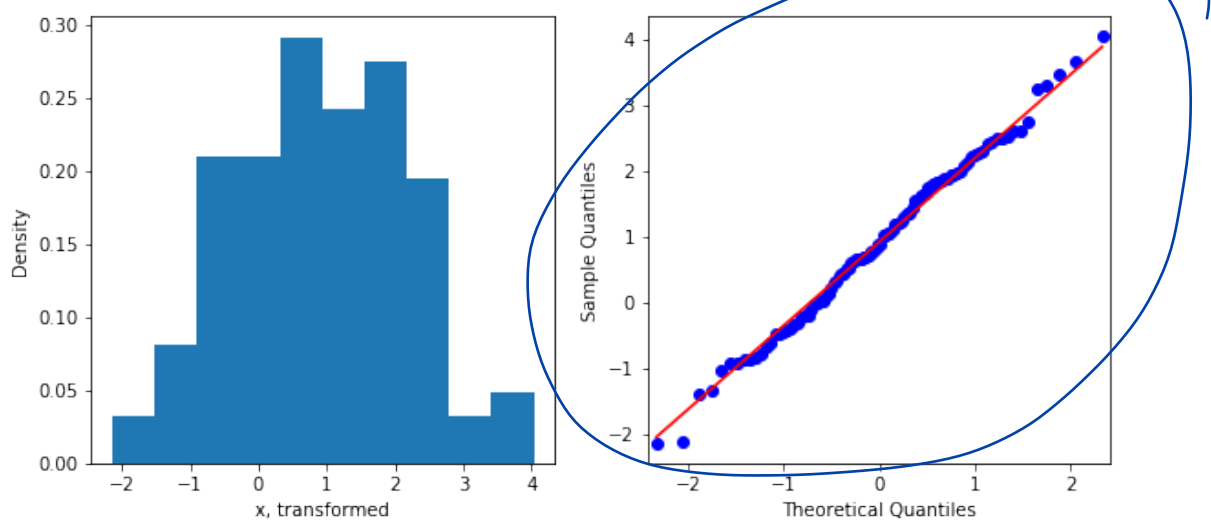
print(lam)
print(CI)
```

```
0.12639088708173513
(-0.02035547883356052, 0.2772495624156657)
```

← 95% CI

```
In [5]: fig, axs = plt.subplots(1, 2, figsize=[10,4.5])
        axs[0].hist(xt,density=True)
        axs[0].set_xlabel("x, transformed")
        axs[0].set_ylabel("Density")

        qqplot(xt,line="s",ax=axs[1])
        plt.show()
```



Comment: It is standard to choose a “nice” value of λ from the 95% confidence interval and use that for the transformation. For example, if 0.5 is in the confidence interval, one would often prefer to use that instead of $\lambda = 0.43$, because the square root transform is easier to interpret. Likewise, if zero is in the interval, the log transform is typically utilized.

Exercise: Consider the volume data below.

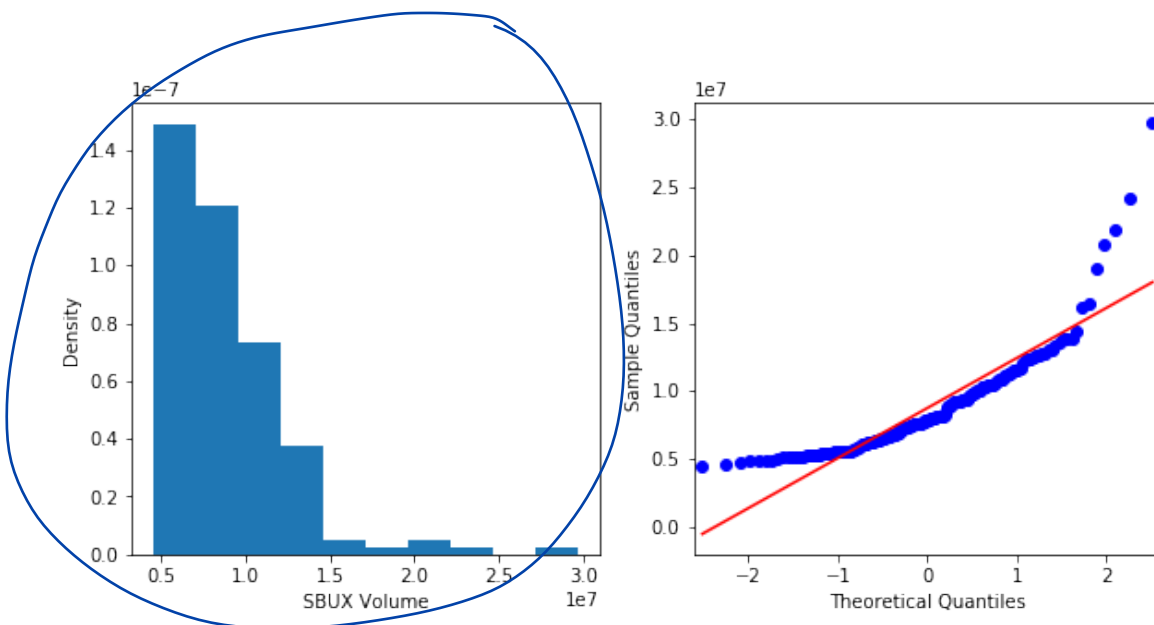
```
In [6]: import yfinance as yf

        SBUX = yf.download("SBUX", start="2019-01-01",
                           end="2019-08-31")

        fig, axs = plt.subplots(1, 2, figsize=[10,4.5])
        axs[0].hist(SBUX['Volume'],density=True)
        axs[0].set_xlabel("SBUX Volume")
        axs[0].set_ylabel("Density")

        qqplot(SBUX['Volume'],line="s", ax=axs[1])
        plt.show()
```

[*****100%*****] 1 of 1 down



Try the Box-Cox transformation in this case. What happens?

Box-Cox for Regression

The original motivation for this type of transformation was in the context of the linear regression model

$$Y^{(\lambda)} = \beta_0 + \sum_{j=1}^p \beta_j x_j + \epsilon$$

The question posed by Box and Cox was as follows: What choice of λ yields the best fitting model, if we want to assume the errors are i.i.d. Normal?

Through a maximum likelihood procedure, the best λ can be found, and then this choice is used to transform the response prior to fitting the full regression model.

Exercise: The Python function `boxcox()` described above operates on a single vector. How could that be used in the context of this regression problem?

The Y 's are never identically distributed
in regression.

So, it does not make sense to use
`boxcox()` on the response

Calling R from Python

For all of the appeal of Python, R is definitely better suited for certain statistical and machine learning procedures. It is possible to take advantage of these capabilities directly through Python, using the package rpy2.

There are different ways of doing this. Here we will show an approach which allows you to embed chunks of R code within the Jupyter notebook, and pass objects back and forth as needed.

First, load the needed package, and load the interactive extension using the following two commands.

```
In [7]: import rpy2  
        %load_ext rpy2.ipython
```

R does not like variable names with spaces in them.

```
In [8]: SBUX = SBUX.rename(columns=\n                           {"Adj Close": "AdjClose"})
```

The R function `boxcox` that is part of the package `MASS` implements the Box-Cox procedure. Note that here, we only need to look at the plot, so no objects are returned from the R code. To return objects, use the argument `-o` in the first line. We see examples of that later.

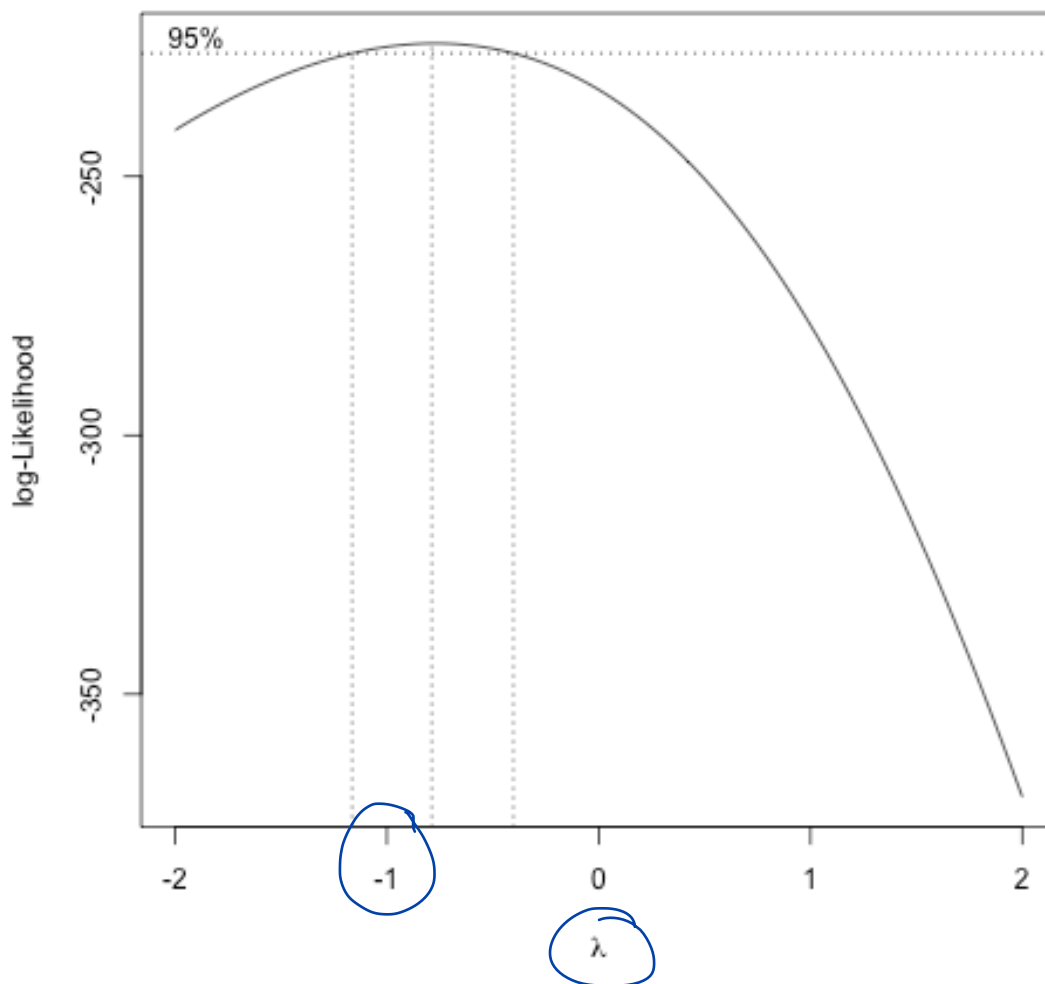
```
In [9]: %%R -i SBUX
```

input
-0
data frame

executed
in R

```
library(MASS) ← load MASS  
boxcox(Volume~AdjClose, data=SBUX)
```

response predictor data frame



Exercise: What conclusion do you draw from this analysis?

Should use $1/y$ as the response

Exercise: Fit the linear model using an appropriate transformation of the response. Assess the results.

The Yeo-Johnson Transformation

The Box-Cox Transformation suffers from the limitation that it can only be applied to strictly positive variables. A generalization that works for any variable is the **Yeo-Johnson Transformation**.

The Yeo-Johnson Power Transformation is

$$y^{(\lambda, \lambda)} = \begin{cases} ((y + 1)^\lambda - 1) / \lambda, & \text{if } \lambda \neq 0, y \geq 0 \\ \log(y + 1), & \text{if } \lambda = 0, y \geq 0 \\ - \left[(-y + 1)^{2-\lambda} - 1 \right] / (2 - \lambda), & \text{if } \lambda \neq 2, y < 0 \\ - \log(-y + 1), & \text{if } \lambda = 2, y < 0 \end{cases}$$


It appears somewhat messy, but the intention is much like for the Box-Cox Transformation: As λ varies, we get a wide range of power and log transformations of the data; the optimal transformation can be sought via maximum likelihood.

In Python this is implemented in `scipy.stats.yeojohnson`.

In []: